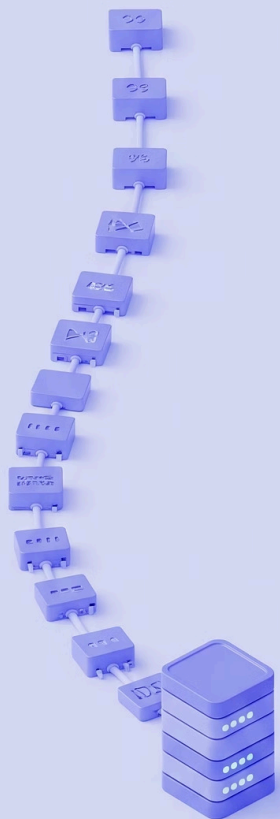




Streaming Infrastructure for Real-Time AI

Building the critical foundation that enables AI to thrive on continuous data flow, delivering low-latency predictions and feedback loops through technologies like **Kafka**, **Flink**, **Pulsar**, and **Kinesis**.

Batch Processing



VS



Real-Time Streaming



Why Streaming for AI?

Speed Requirements

Traditional batch pipelines are simply too slow for critical applications like fraud detection, ad serving, and real-time personalization

Event-Driven Architecture

Real-time AI requires systems that respond immediately to events rather than processing data in scheduled intervals

Complete Pipeline

Streaming infrastructure instantly ingests data, processes and enriches features, then delivers to inference APIs—all in milliseconds

Core Components of Streaming Infrastructure

1

Event Brokers

Technologies like **Kafka**, **Pulsar**, and **Kinesis** serve as the backbone of streaming systems, reliably routing events to consumers

2

Stream Processors

Flink, **Spark Streaming**, and **Materialize** transform raw events into meaningful data through stateful computations

3

Feature Stores

Systems like **Feast** and **Tecton** maintain consistency between online (low-latency) and offline (training) environments

4

Serving Layer

Optimized inference engines deliver predictions with minimal latency using the freshest possible data

Event Broker: The Central Nervous System

Critical Functions:

- Ingests diverse event types: user clicks, financial transactions, IoT sensor readings, and application logs
- Guarantees **event ordering** within partitions for accurate sequence processing
- Ensures **durability** through persistent storage and replication
- Supports **replay capabilities** for recovery and reprocessing historical data

Popular choices: **Kafka** and **Pulsar**



Stream Processing: Real-Time Transformations



Example: A transaction event is enriched with the user's recent login locations, device fingerprints, and purchase history—all computed in-flight before reaching the fraud detection model.

Feature Store Integration

The Real-Time Challenge

Online feature stores must serve **fresh features in milliseconds** to support real-time inference, while maintaining consistency with offline training data.

Integration Patterns:

- Streaming pipelines **push updates directly** to online stores
- Feature computation handled **in-stream** for maximum freshness
- Materialized views kept updated for instant access

Real-World Examples

Ad Auction

User's last click feature available in <100ms for real-time bidding decisions

Fraud Detection

Location and device information updated in real-time to catch anomalies immediately

Latency Budget in Streaming AI Pipelines



10%

Broker Publish/Consume

<10ms for message routing through the event broker



30%

Stream Processing

10–30ms for joins, aggregations, and transformations



15%

Feature Store Lookup

5–15ms to retrieve pre-computed features



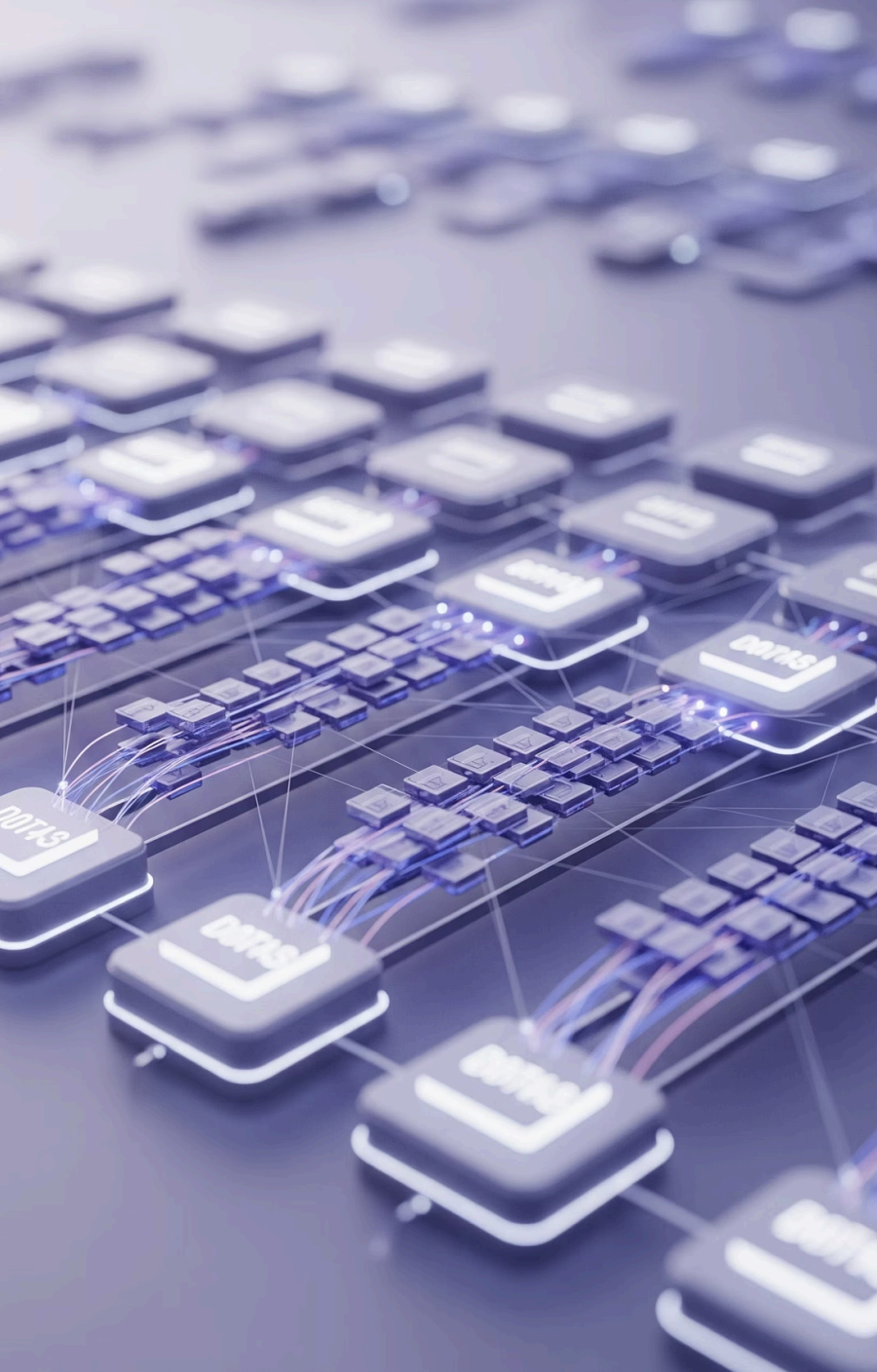
40%

Model Inference

10–40ms for prediction generation

Critical Threshold

End-to-end pipeline must stay under **100 milliseconds** for truly real-time AI applications like ad serving, fraud detection, and interactive recommendations.



Scaling Streaming Infrastructure



Partitioning

Divide data streams into partitions that can be processed independently and in parallel



Autoscaling

Dynamically adjust consumer instances using Kubernetes to match incoming data volume



Multi-Region

Replicate data across geographic regions to support global applications with local processing

Effective backpressure handling is essential to prevent message drops during load spikes. Properly configured systems will slow down producers rather than lose data.

Reliability in Streaming AI Systems

Critical Requirements

Exactly-once Semantics

Prevent duplicate processing that could lead to incorrect charges, multiple fraud flags, or skewed analytics

Replayability

Recover from outages by reprocessing event logs from a specific point in time

Dead-letter Queues

Isolate malformed events for investigation without blocking the main pipeline

Key Monitoring Metrics

- **Consumer lag:** How far behind real-time is processing?
- **Throughput:** Events/second through each component
- **Error rate:** Percentage of events failing processing
- **Processing latency:** Time from event creation to completion
- **Resource utilization:** CPU, memory, network I/O



Real-World Example: Real-Time Advertising



Process Flow

1. User interaction generates click event
2. Event lands in Kafka within 2-5ms
3. Flink joins with user context & history (10-15ms)
4. Feature store updates user profile (5ms)
5. Model inference generates bid recommendation (15-20ms)

Technical Requirements

- Total latency budget: <50ms to meet ad auction deadlines
- Throughput: Millions of events per second during peak hours
- Availability: 99.99% uptime required (5 minutes downtime/month max)
- Global deployment with region-specific compliance rules

Real-World Example: Fraud Detection

Transaction Event

Credit card swipe, online purchase, or money transfer generates an event with amount, location, merchant, device details

Streaming Enrichment

Kafka routes to Flink for real-time enrichment with user history, location patterns, and device fingerprints

Rule Evaluation

Business rules checked in parallel with ML model scoring to catch known fraud patterns

Model Scoring

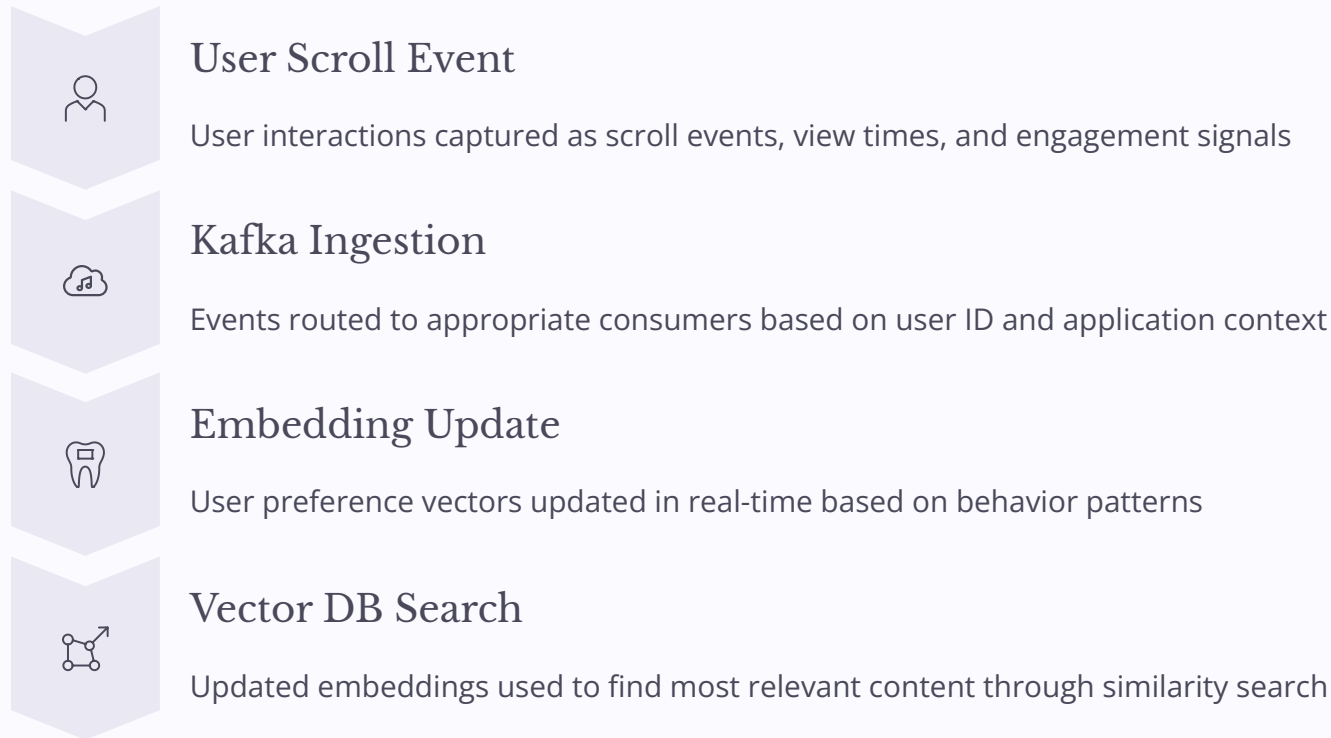
ML model generates fraud probability score using fresh features from the stream

The entire pipeline delivers approve/deny decisions in **under 40ms**, with continuous feedback loops that improve model accuracy over time.

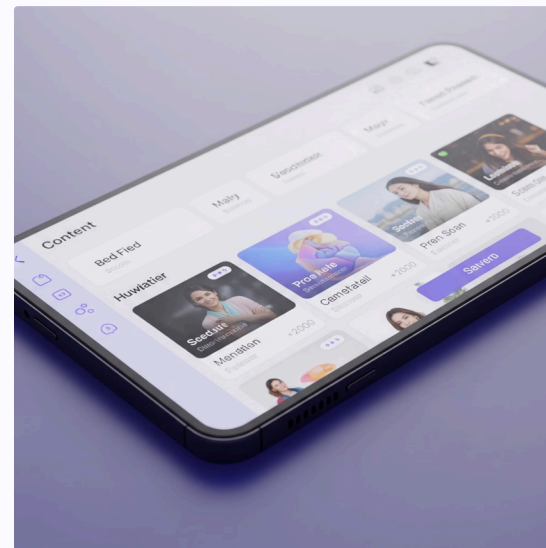


Real-World Example: Real-Time Personalization

Content Feed Personalization Flow



This architecture enables **real-time adaptation** to user behavior, dramatically improving engagement metrics compared to batch-updated recommendations.



"Our recommendation relevance improved 37% after moving from hourly batch updates to streaming personalization."

Streaming Tools Comparison



Apache Kafka

- **Strengths:** Mature ecosystem, strong durability guarantees, widespread adoption
- **Weaknesses:** More complex to operate, higher end-to-end latency in some cases
- **Best for:** Enterprises with experienced teams, high-throughput applications

Apache Pulsar

- **Strengths:** Multi-tenancy, geo-replication, generally lower publish latency
- **Weaknesses:** Smaller ecosystem, fewer managed service options
- **Best for:** Global applications, multi-tenant environments



AWS Kinesis

- **Strengths:** Fully managed, tight AWS integration, minimal operational overhead
- **Weaknesses:** Vendor lock-in, more expensive at scale, throughput limitations
- **Best for:** AWS-native architectures, teams wanting operational simplicity



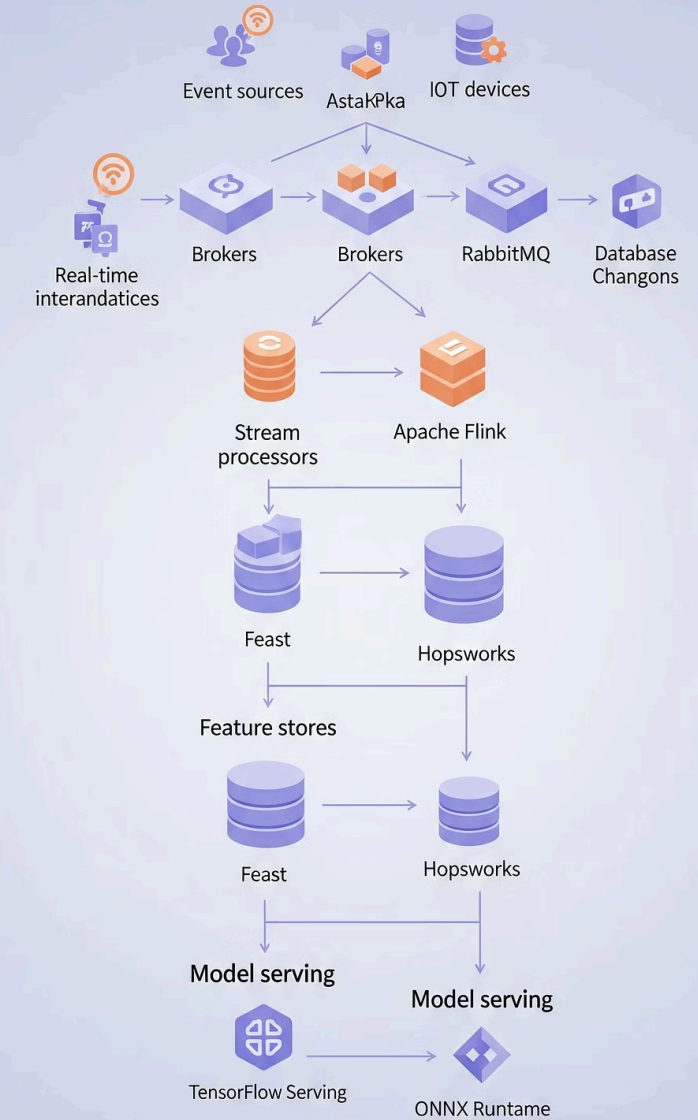
Flink vs Spark Streaming

- **Flink:** Lower latency, true streaming semantics, sophisticated windowing
- **Spark:** Unified batch+stream API, easier adoption for Spark users
- **Decision factor:** Latency requirements and existing team expertise

Streaming Infrastructure Architecture



"Streaming infrastructure forms the backbone that powers real-time AI decisions across the enterprise."



Streaming Infrastructure: The Spine of Real-Time AI

Key Components

- **Event brokers:** Kafka, Pulsar, Kinesis
- **Stream processors:** Flink, Spark Streaming
- **Feature stores:** Online/offline consistency layer
- **Serving layer:** Low-latency inference endpoints

Critical Requirements

- End-to-end latency under 100ms
- Horizontal scalability for traffic spikes
- Exactly-once processing guarantees
- Monitoring and observability at each stage

Next Steps

- Assess current infrastructure against real-time requirements
- Identify highest-value AI use cases requiring streaming
- Start with pilot implementation on non-critical workloads
- Build operational expertise before scaling to production